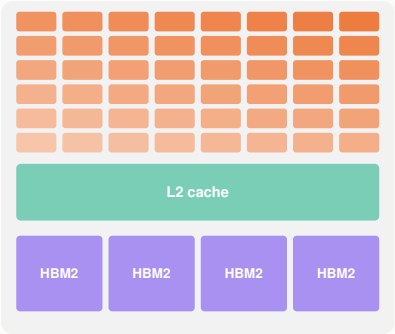
LECTURE NOTES · HARDWARE, 2026

Introduction to GPU Architectures

How graphics chips became throughput processors: the fixed-function pipeline, programmable shaders and GPGPU, the unified G80 design and CUDA, SIMT execution and latency hiding, the memory system and the roofline, the Tesla to Volta generations, HBM2, NVLink and tensor cores, and the systems built from them.

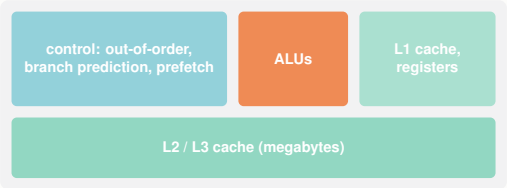


80 streaming multiprocessors, one shared L2 cache, four stacks of HBM2: the floorplan of a throughput processor (Volta, 2017)

A GPU is a throughput processor

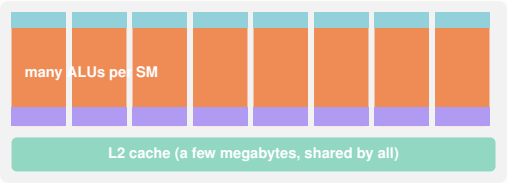
Spend the transistors on arithmetic and on threads in flight, not on making one thread fast.

a CPU core spends its area on latency



a GPU spends it on ALUs and on warps waiting their turn

thin control register files and shared memory per SM



5,120

FP32 lanes on V100

900 GB/s

HBM2 bandwidth, V100

The contract

Many threads, not fast threads. A V100 keeps up to 163,840 threads resident and switches between groups of 32 every cycle; a thread waiting on memory costs nothing while others run (slide 12).

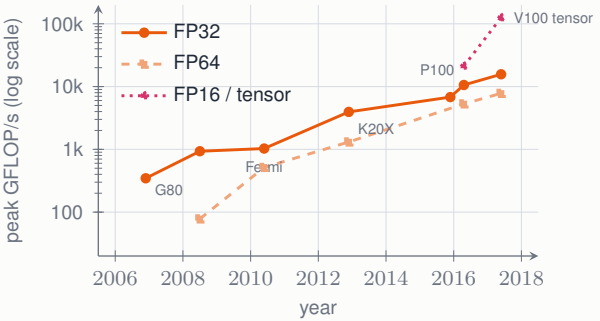
Regular work. One instruction runs on 32 data items at once (SIMT, slide 11); branches that split the group cost time.

Bandwidth is the budget. Arithmetic is cheap, data movement is not; the roofline (slide 15) says which kernels fit. A host CPU launches the kernels (slide 9).

Three numbers: FLOP/s, bytes per second, watts

Peak arithmetic grew about 45× from G80 to V100 while board power doubled; memory bandwidth grew about 10×.

NVIDIA compute GPUs, vendor peaks† (G80 counted MAD-only)



86 → 900

GB/s, G80 to V100

155 → 300

watts, G80 to V100

What the curves say

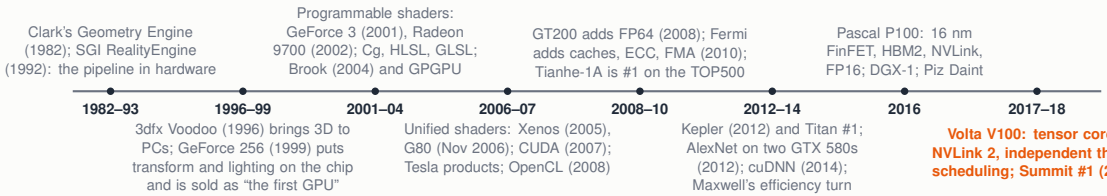
Arithmetic scaled with transistors: 681M (G80) to 21.1B (GV100) at about twice the board power, by moving to 16 and 12 nm and adding lanes, not clock speed (1.35 to 1.53 GHz).

Bandwidth lagged: 0.25 bytes per FLOP on G80, 0.06 on V100 (derived); each generation is more compute-bound, which is why caches, shared memory and HBM2 matter (slides 14–15).

Precision became a dial: FP64 came in 2008 for HPC, FP16 and tensor cores in 2016–17 for deep learning.

A timeline: from the Geometry Engine to Volta

Thirty-five years in four moves: fixed function, programmable shaders, unified compute, specialized compute.



What the order tells you

Each move was forced by the one before. Fixed pipelines ran out of effects to add, so stages became programmable; programmable stages sat idle in turn, so they were unified; a unified array of ALUs was a general processor, so people programmed it; and once the programmers were deep-learning researchers, the ALUs specialized again (tensor cores). This deck follows that order.

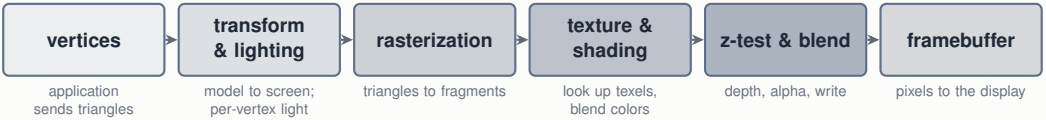
What stayed the same

The job. From the RealityEngine to Volta the hardware exploits one fact: the work is a huge number of independent, identical computations (pixels, vertices, matrix elements) with regular memory access. Every design choice on the following slides (SIMT, warps, wide memory, small caches, many threads in flight) is a way to exploit that regularity cheaply.

The fixed-function graphics pipeline

Vertices in, pixels out: a hardware assembly line whose stages were fixed in silicon until 2001.

the pipeline, as the GeForce 256 (1999) implemented it



Why a pipeline, and why hardware

Every vertex goes through the same transform and every fragment the same shading; nothing depends on its neighbor: identical, independent work, which hardware parallelizes by replicating units and keeping all stages busy at once. Clark's Geometry Engine (1982) did the vertex math in a custom chip; SGI's RealityEngine (1992) built the whole pipeline from boards of geometry processors and raster chips; 3dfx's Voodoo (1996) put rasterization and texturing on one consumer card.

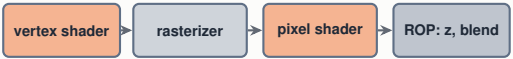
GeForce 256: the word "GPU"

NVIDIA's GeForce 256 (1999) moved transform and lighting from the CPU onto the chip, so the whole pipeline ran on one die, and marketed it as "the world's first GPU"[†]. The definition was a marketing one, but the architecture behind it is real: a fixed set of stages, each a wide, specialized unit, fed by a memory system built for texture reads. ATI followed with the Radeon (2000). What neither could do yet was run a program a developer wrote; effects were chosen from a fixed menu.

Programmable shaders: a processor hiding in each stage

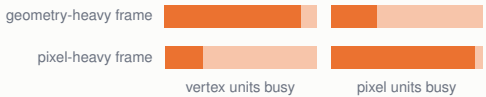
2001–2004: vertex and pixel stages become small SIMD processors; separate units per stage leave silicon idle.

the pipeline with programmable stages (DirectX 9 era, 2002–04)



red: runs a developer's program per vertex or per pixel; blue: still fixed function

load imbalance with separate units (illustrative)



whichever stage is light, its units idle; the fix is one pool of processors that any stage can use (slide 8)

The steps

GeForce 3 (2001): a programmable vertex engine; the developer writes a short assembly program that runs per vertex (Lindholm et al., SIGGRAPH 2001).

Radeon 9700 (2002): floating-point pixel shaders for DirectX 9; GeForce FX and 6800 (2003–04) add longer programs, branching and texture reads in the vertex stage.

Languages: Cg, HLSL and GLSL (2002–04) make shaders C-like.

What a shader unit is: a small SIMD processor running the same program on many vertices or pixels, with a texture unit as its only way to read memory and the framebuffer as its only way to write. Researchers noticed that this is a parallel computer with an odd interface (slide 7).

GPGPU: computing with pixels

2003–2006: arrays became textures, kernels became fragment shaders, and a graphics API hid a parallel computer.

the GPGPU mapping (Brook for GPUs, 2004)

array $A[i, j]$	→	**texture**
loop over elements	→	**draw a full-screen quad**
kernel body	→	**fragment shader**
output array	→	**render-to-texture**
read back	→	**copy to host**

every output pixel is one kernel invocation; the rasterizer is the loop; texture fetches are the only reads

What it could do

Dense linear algebra, FFTs, fluid simulation, ray tracing and sorting all ran faster than on CPUs of the day; Owens et al.'s survey (2007) catalogs several hundred papers. Brook (Buck et al., 2004) and Sh gave the pattern a language: streams and kernels compiled to shaders. This is the community that later wrote the first CUDA libraries.

What it could not do

No scatter: a fragment writes only to its own pixel, so histograms and sparse updates needed tricks.

No on-chip sharing: no shared memory, no synchronization between fragments; any communication went through the framebuffer and another pass.

Odd arithmetic: 16- or 24-bit floats on some parts, no integers, no IEEE rounding guarantees, no double precision.

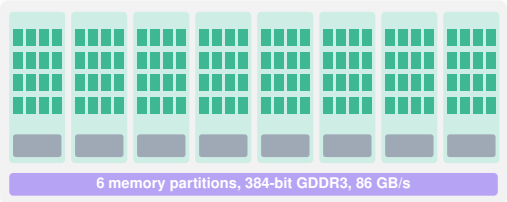
Fixed resources: register counts, program lengths and output counts capped by the graphics API; the driver, not the programmer, decided scheduling.

The lesson for 2006: keep the wide SIMD arrays and the texture memory system; remove the graphics API from between the programmer and the hardware (slide 8).

G80 (2006): one unified array of processors, and CUDA

The GeForce 8800 GTX replaced separate vertex and pixel units with 128 identical processors, then got a C compiler.

G80: 8 texture/processor clusters, each 2 SMs of 8 streaming processors



green: 128 scalar FP32 processors in 16 SMs at 1.35 GHz; blue: texture units. Each SM: 8 SPs, 16 KB shared memory, 8,192 registers, a warp scheduler; 681M transistors, 470 mm^2, 90 nm, about 150 W typical

Why unify

One pool of processors serves whichever stage is busy, so the load imbalance of slide 6 disappears, and the same pool runs a non-graphics program directly. ATI's Xenos in the Xbox 360 (2005) did it first; G80 did it at PC scale with a scalar ISA.

CUDA (2007) and what followed

A C compiler for the array: CUDA (public beta February 2007) exposes the SMs as a hierarchy of threads, blocks and grids (slide 9) with shared memory and barriers; no textures, no graphics API.

Tesla: the C870 (June 2007) is the same chip sold with no display output, for computing; the brand name of NVIDIA's compute line through V100.

The competition answers: ATI's Close To Metal (2006) and Stream; the Khronos OpenCL 1.0 standard (December 2008) with the same model and vendor-neutral names (work-items, work-groups).

Why it stuck: the model matched the hardware (slide 10), the hardware shipped in every gaming PC, and the libraries (cuBLAS, cuFFT, later cuDNN) made the common cases fast without writing kernels.

The CUDA programming model: grids of blocks of threads

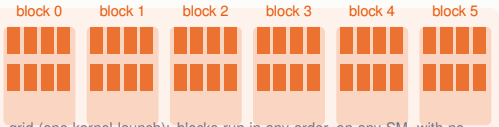
Write the program for one thread; launch tens of thousands; let the hardware schedule blocks onto SMs.

SAXPY, $y \leftarrow ax + y$, as a kernel

```
__global__ void saxpy(int n, float a, float *x, float *y) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) y[i] = a * x[i] + y[i];
}

// host: 256 threads per block, enough blocks for n
saxpy<<<(n + 255) / 256, 256>>>(n, 2.0f, d_x, d_y);
```

the launch: a grid of blocks, each a block of threads



grid (one kernel launch): blocks run in any order, on any SM, with no communication between them

threads in a block share `__shared__` memory and can wait at `__syncthreads()`; threads in different blocks cannot

Memory spaces, as the programmer sees them

Registers: per thread, allocated by the compiler; the largest and fastest storage on the chip (slide 10).

Shared memory: per block, on the SM, programmer-managed; 16 KB on G80, up to 96 KB on Volta; the tool for reuse and for cooperation within a block.

Global memory: the GPU's DRAM, visible to every thread and to the host through copies (or, from Pascal, through page-faulting unified memory); latency of hundreds of cycles.

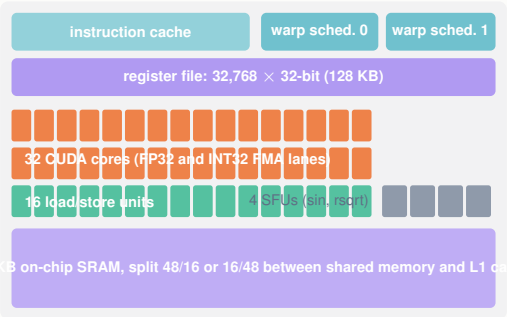
Constant and texture: read-only, cached paths inherited from graphics; local memory: register spills, in DRAM.

Why this shape: it is the hardware (slide 10) with the names changed. Nickolls et al. (2008) is the paper; Kirk and Hwu's textbook is the course.

Mapping the model to the chip: blocks to SMs, threads to warps

A streaming multiprocessor runs several blocks at once; its scheduler issues one instruction for 32 threads at a time.

one Fermi SM (2010): the unit the rest of the deck is about



up to 8 blocks and 48 warps (1,536 threads) resident per SM; 16 SMs per GF100 (Wittenbrink et al. 2011)

The mapping

Block → SM. A block runs entirely on one SM, which is why its threads can share memory and synchronize. The GigaThread engine hands blocks to SMs as resources free up; extra blocks queue.

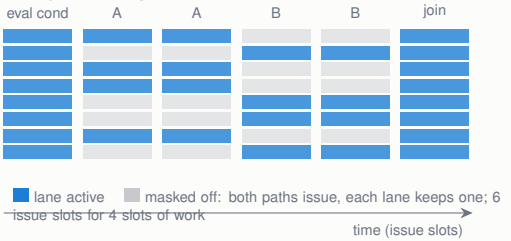
32 threads → warp. The SM groups consecutive threads of a block into warps and schedules warps, not threads: one instruction per warp per issue, over 32 lanes (slide 11).

Occupancy. Registers, shared memory and warp slots are the three resources a block consumes; the ratio of resident warps to the maximum is the occupancy, and it is what hides latency (slide 12). A kernel using 64 registers per thread fills a Fermi register file with 512 threads, a third of the slots. The picture holds through Volta with the counts changed (slide 18).

SIMT: SIMD hardware with a thread's face

One instruction, 32 lanes, each lane its own thread: branches are allowed and divergent branches are paid for in time.

a warp executing `if (x[i] > 0) A; else B;` (8 lanes shown)



Three ways to be wide

SIMD: one instruction, one register of n lanes; the compiler packs the data. **MIMD:** independent instruction streams, each with its own fetch and decode. **SIMT:** SIMD execution with per-lane program counters simulated by masks; each lane is a scalar thread with its own addresses and branches.

What it costs and what it buys

Divergence: a warp whose lanes take k paths through a branch runs them serially, worst case $32\times$ slower; loops with different trip counts per lane do the same. Fix: group work so that neighboring threads agree.

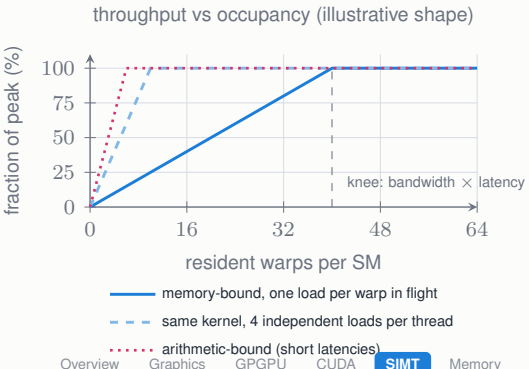
Memory divergence: 32 addresses per load; if they fall in one 128-byte segment it is one transaction, if they scatter it is up to 32 (slide 14).

Reconvergence: through Pascal a warp has one program counter and a stack of branch targets, so lanes on one path cannot wait for lanes on the other (a spin lock across a warp deadlocks). Volta gives each thread its own program counter (slide 24).

What it buys: a scalar programming model on SIMD hardware and one instruction fetch per 32 operations (slide 13).

Latency hiding: enough threads in flight to cover the memory

A warp that waits on DRAM is parked and another issues; Little's law says how many must be in flight to keep DRAM bus



The arithmetic

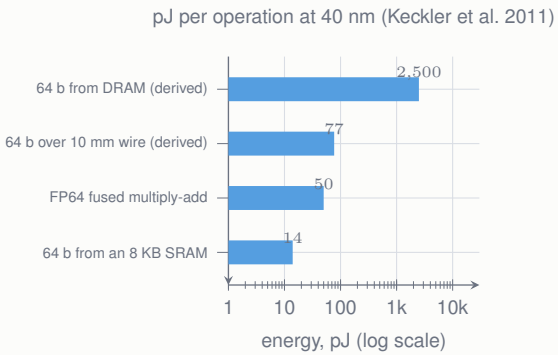
Little's law: bytes in flight = bandwidth × latency. V100: 900 GB/s × 0.27–0.75 μs of DRAM latency (Jia et al. 2018: 375 to 1,029 cycles on a 1.38 GHz PCIe card, TLB hit to miss) = 245–670 KB outstanding, 3–8 KB per SM: 24–66 128-byte loads in flight per SM (derived).

Two ways to get them: more warps (thread-level parallelism), or more independent loads per thread before the first use (instruction-level parallelism). Volkov (2010): fewer warps with more ILP can beat full occupancy, because each thread then keeps more registers (65,536 per SM is 32 per thread at full occupancy).

Why the CPU does it differently: an out-of-order core finds independent loads in one thread's instruction window with large, power-hungry structures; the GPU asks for many threads and keeps the hardware simple.

Energy is the real constraint: why the design is what it is

After Dennard scaling ended (about 2005), performance per watt decided the design; moving data costs more than math



What follows from the energies

Amortize instructions: one fetch and decode per 32 lanes (SIMT) instead of per operation; the control energy that dominates a scalar core becomes a rounding error.

Keep data close: huge register files (256 KB per SM on Pascal and Volta, more than the L1), programmer-managed shared memory, reuse within a block: a 64-bit DRAM access costs about fifty FMAs, and the ratio widens with every process node.

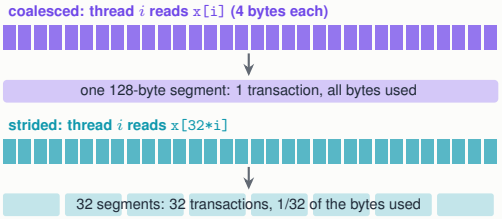
Wide, slow, many: once clock speed stopped paying, the GPU ran more lanes at 0.7–1.5 GHz instead of fewer at 3–4 GHz; at fixed power only part of a chip can be lit (“dark silicon”, Esmaeilzadeh et al. 2011), so specialization wins.

Move memory closer, specialize: HBM2 on an interposer shortens and widens the DRAM wires (slide 20); a tensor core does 64 FMAs per clock (slide 23).

The memory system: wide DRAM, coalescing, small caches

Bandwidth comes from wide buses and many requests in flight; the programmer turns 32 addresses into one transaction

coalescing: a warp's 32 loads become memory transactions



What changed, 2006 to 2017

DRAM: GDDR3 at 86 GB/s on a 384-bit bus (G80); GDDR5 at 144–288 GB/s (Fermi, Kepler); HBM2 at 720–900 GB/s on a 4,096-bit bus (Pascal, Volta): wider and slower per pin, not faster.

Caches: none for global loads on G80; Fermi adds an L1 per SM and a 768 KB L2 with ECC; Volta has 6 MB of L2.

Rules the hardware imposes

Granularity: a warp's 32 addresses are sorted into the 32-byte sectors and 128-byte lines they touch; each becomes one request. Consecutive threads, consecutive addresses; structure-of-arrays beats array-of-structures.

Shared memory: 32 banks, 4 bytes wide; 32 lanes on 32 banks take one cycle; two lanes on one bank serialize. Pad arrays by one word to break strides.

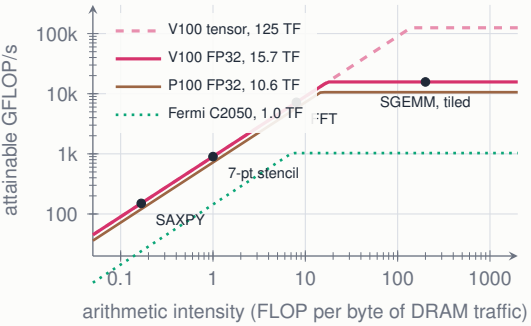
Caches are small per thread: 6 MB of L2 over 163,840 resident threads is 38 bytes each (V100, derived). Reuse must be explicit, in registers and shared memory, which is what tiled matrix multiply does.

ECC: Fermi brought it; HBM2 made it nearly free; the supercomputers of slide 26 required it.

The roofline: which kernels a GPU can feed

Attainable FLOP/s is the smaller of peak arithmetic and bandwidth times intensity; most kernels sit on the slope.

rooflines from vendor peaks (derived); kernel points are typical intensities



Reading the plot

Ridge point = peak ÷ bandwidth: 17 FLOP/B for V100 FP32, 139 for its tensor cores, 7 for Fermi (derived); left of it a kernel is bandwidth-bound.

SAXPY: 2 FLOPs per 12 bytes; on V100 it tops out near 150 GFLOP/s, 1% of peak, which is correct, not a bug.

Matrix multiply reuses each loaded element $O(n)$ times from shared memory and registers, so its intensity is set by the tile; it is the common kernel that reaches the roof, which is why cuBLAS and cuDNN defined what GPUs are for.

Every generation raised the ridge: 4 FLOP/B on G80, 17 on V100, 139 with tensor cores; kernels that fit in 2008 are bandwidth-bound in 2017 (Williams et al. 2009).

Tesla to Fermi (2008–2010): double precision, caches, ECC

GT200 made the GPU usable for science; Fermi made it a general processor and put it at the top of the TOP500.

GT200 (GTX 280, Tesla C1060; June 2008)

240 SPs in 30 SMs, 1.4B transistors on 65 nm, a 576 mm^2 die. New: one FP64 unit per SM (78 GFLOP/s double against 933 single on the C1060), twice the registers, 4 GB of GDDR3 at 102 GB/s, shared memory atomics. Double precision at a twelfth of single was slow but it existed, and the first HPC ports (molecular dynamics,

Fermi, GF100 (Tesla C2050/C2070; 2010)

512 CUDA cores in 16 SMs of 32 (the C2050 shipped 448), 3.0B transistors on 40 nm. New: a cache hierarchy (64 KB L1/shared per SM, 768 KB L2), ECC everywhere, IEEE 754-2008 FMA, FP64 at half the FP32 rate (515 against 1,030 GFLOP/s), a unified 40-bit address space with C++ pointers, two warp schedulers per SM, concurrent kernels (Wittenbrink et al. 2011).

Why Fermi is the pivot

It settled the SM. The Fermi SM of slide 10 is the template every later generation varied in counts and never replaced.

It made GPUs eligible for supercomputers. ECC and real FP64 were the conditions: Tianhe-1A (7,168 Tesla M2050s) was #1 on the TOP500 in November 2010; TSUBAME 2.0 (M2050) and Nebulae (C2050) used the same Fermi die.

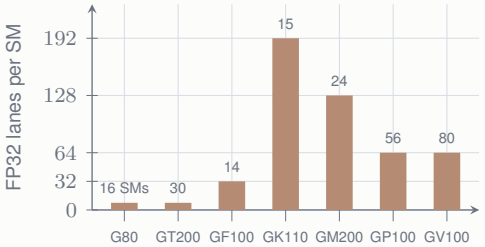
It priced the general-purpose features. Caches, ECC and FP64 cost area and power that games do not use; from Kepler on, the compute dies (GK110, GP100, GV100) carried them and the consumer dies did not.

The GeForce GTX 480 (March 2010), the consumer Fermi, ran at 250 W and taught the lesson that produced Maxwell (slide 17).

Kepler and Maxwell (2012–2015): wider SMs, then efficiency

Kepler put 192 lanes in an SM and 18,688 GPUs in Titan; Maxwell cut back to 128 lanes and doubled perf per watt†.

FP32 lanes per SM (bars) and SMs enabled on the named product (labels above the bars)



lanes per SM went up and came back down; total lanes and transistors per die grew every generation, while transistors per lane fell from Fermi (5.9M) to Kepler (2.5M) and rose again with Pascal (4.0M)

Kepler, GK110 (Tesla K20X 2012, K40 2013)

15 SMX of 192 lanes (2,880), 7.1B transistors on 28 nm; 64 FP64 units and 65,536 registers per SMX. K40: 4.3 TFLOP/s single, 1.4 double at base clock, 12 GB GDDR5 at 288 GB/s, 235 W. New: dynamic parallelism, Hyper-Q (32 hardware queues), GPUDirect RDMA. Titan's 18,688 K20Xs were #1 in November 2012; the K80 (2014) put two GK210s on one board.

Maxwell, GM204 / GM200 (2014–15)

128 lanes per SM in four quadrants with their own schedulers and register files; a larger L2 (2–3 MB); about twice Kepler's perf per watt on the same 28 nm process by keeping each lane busier†. FP64 cut to 1/32 of FP32: Maxwell served GeForce and training (Tesla M40, 6.8 TFLOP/s), not HPC, while GP100 was built.

The generations in one table

Seven compute dies, 2006–2017: the counts change by an order of magnitude; the structure does not.

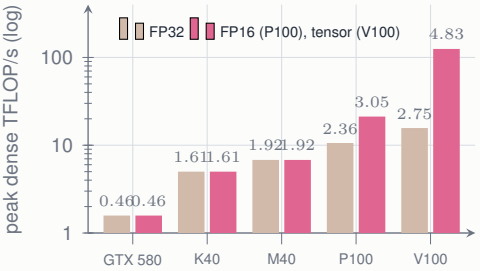
die (product)	**G80** 8800 GTX	**GT200** C1060	**GF100** C2050	**GK110** K40	**GM200** M40	**GP100** P100	**GV100** V100
year	2006	2008	2010	2013	2015	2016	2017
process (nm)	90	65	40	28	28	16	12
transistors (B)	0.68	1.4	3.0	7.1	8.0	15.3	21.1
die area (mm^2)	470	576	529	551	601	610	815
SMs $\times$ FP32 lanes	16×8	30×8	14×32	15×192	24×128	56×64	80×64
FP32 TFLOP/s	0.35	0.93	1.03	4.3	6.8	10.6	15.7
FP64 TFLOP/s	—	0.08	0.52	1.4	0.21	5.3	7.8
FP16 / tensor TFLOP/s	—	—	—	—	—	21.2	125
memory	GDDR3	GDDR3	GDDR5	GDDR5	GDDR5	HBM2	HBM2
capacity (GB)	0.75	4	3	12	12	16	16 / 32
bandwidth (GB/s)	86	102	144	288	288	720	900
L2 (KB)	—	—	768	1,536	3,072	4,096	6,144
registers per SM (KB)	32	64	128	256	256	256	256
shared memory per SM (KB)	16	16	48	48	96	64	96
TDP (W)	155	188	238	235	250	300	300
host link	PCIe 1	PCIe 2	PCIe 2	PCIe 3	PCIe 3	NVLink	NVLink 2

vendor specifications for the named products[†] (G80 as the GeForce 8800 GTX; its 155 W is a database figure, Lindholm et al. give 150 W typical); G80 FP32 counts MAD only (518 with the co-issued MUL) while the C1060's 933 includes it (622 without); GF100 column is the C2050 as shipped (14 of 16 SMs); K40 at base clock (5.0 / 1.7 with boost); die areas for GT200 and GF100 are third-party measurements; registers in KB are 32-bit entries $\times$ 4; shared memory is the maximum configuration.

Deep learning arrives (2012–2016) and the architecture answers

AlexNet trained on two GeForce cards; five years later the compute die had a unit for nothing but matrix multiply.

he fastest precision a trainer could use, per generation (boost clocks)†



What happened

2012: Krizhevsky, Sutskever and Hinton train AlexNet on two GTX 580s (3 GB each, the network split across them) in five to six days and win ImageNet by a wide margin; convolution is matrix multiply.

2014: cuDNN (Chetlur et al.) gives every framework the same tuned convolutions; Caffe, Torch, Theano, then TensorFlow build on it. Training moves from GeForce to Tesla (K40, K80, M40).

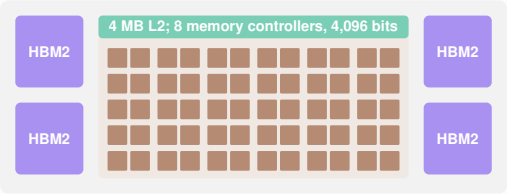
What the workload asked for: dense matrix throughput, memory for activations, fast links between GPUs for data-parallel training, and tolerance of low precision.

What the hardware answered: Pascal, FP16 at twice the FP32 rate, HBM2, NVLink (slide 20); Volta, tensor cores at eight times the FP32 rate (slide 23); mixed-precision training (Micikevicius et al. 2017) to make FP16 safe.

Pascal GP100 (2016): 16 nm FinFET, HBM2, NVLink

The first compute die built for deep learning as well as HPC: 15.3B transistors, 720 GB/s, FP16 at double rate, NVLink.

the GP100 package: die plus four HBM2 stacks on a silicon interposer (CoWoS)



6 GPCs of 10 SMs (56 enabled); 610 mm^2 die on about 1,000 mm^2 of interposer; the stacks put thousands of short, slow wires where GDDR5 had hundreds of fast ones: 2.5× the M40's bandwidth for less energy per bit

15.3B
transistors, 16 nm FinFET

720 GB/s
16 GB HBM2, four stacks

What Pascal added†

The SM: 64 FP32 lanes in two halves, 32 FP64 units (half rate, 5.3 TFLOP/s), 256 KB of registers, 64 KB shared memory: more, smaller SMs than Maxwell.

FP16 at 2×: packed `half2` arithmetic, 21.2 TFLOP/s, the first explicit concession to deep learning.

NVLink 1.0: four 40 GB/s links per GPU, 160 GB/s against 32 for PCIe 3.0 x16; GPU to GPU, and GPU to POWER8 (slide 25).

Unified memory with page faulting: one 49-bit address space for CPU and GPU; pages migrate on demand.

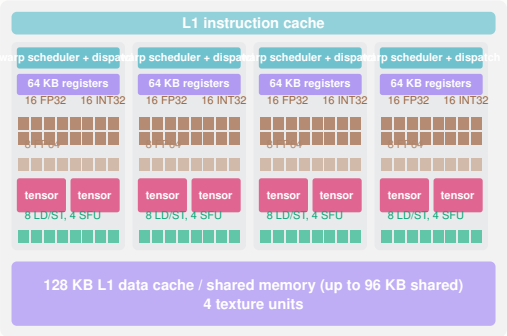
Compute preemption at instruction granularity.

Products: Tesla P100 (April 2016; SXM2 300 W, PCIe 250 W), the DGX-1 with eight, Piz Daint's upgrade (slide 26).

Reading a Pascal or Volta SM: four schedulers, many lanes

The SM is partitioned into quarters, each with its own scheduler and registers; the whole shares L1 and shared memory.

one Volta SM (Choquette et al. 2018)



per SM: 64 FP32, 64 INT32, 32 FP64 lanes, 8 tensor cores, 256 KB registers, 2,048 threads; 80 such SMs on a V100

How to read it

Four quarter-SMs: each scheduler issues one warp instruction per clock to 16 lanes, so a 32-thread warp instruction takes two clocks; four quarters give the 64 lanes per clock the peak numbers assume. Maxwell introduced the partitioning, Pascal halved the SM to 64 lanes, Volta kept 64 and added units.

Separate INT32 lanes (Volta): address arithmetic and loop counters run beside floating point.

Registers above cache: 256 KB of registers over 128 KB of L1/shared; the register file is the real cache, and occupancy is the register budget (slide 12).

Peak arithmetic, derived: 80 SMs $\times$ 64 lanes $\times$ 2 FLOP (FMA) $\times$ 1.53 GHz $=$ 15.7 TFLOP/s FP32; 80 $\times$ 32 $\times$ 2 $\times$ 1.53 $=$ 7.8 FP64.

Volta GV100 (2017): the largest die, and a new kind of unit

21.1 billion transistors on 815 mm^2, near the reticle limit; 80 SMs, 640 tensor cores, 900 GB/s of HBM2, NVLink 2.

21.1B
transistors, 12 nm FFN

815 mm^2
die area, near reticle limit

125 TF
tensor FP16, FP32 accumulate

300 GB/s
NVLink 2.0, six links

The chip[†]

84 SMs on the die, 80 enabled: 5,120 FP32 lanes, 2,560 FP64, 640 tensor cores; 6 MB L2; 16 GB (2018: 32 GB) of HBM2 on four stacks at 900 GB/s; 20 MB of register files, more than the L2. At 1.53 GHz boost: 15.7 TFLOP/s FP32, 7.8 FP64, 125 tensor. TSMC 12 nm FFN, a tuned 16 nm process; 815 mm^2 is near the reticle limit. Products: Tesla V100 SXM2 (300 W) and PCIe (250 W; 7 / 14 / 112), DGX-1V, DGX Station.

What is new, in order of importance

Tensor cores (slide 23): a matrix unit in each quarter-SM, eight times the FP32 rate on FP16 inputs.

Independent thread scheduling (slide 24): a program counter and stack per thread.

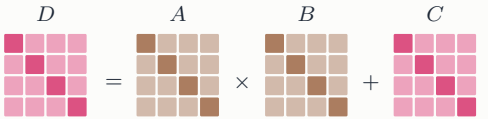
The SM (slide 21): separate INT32 lanes, a 128 KB L1/shared array, four schedulers.

NVLink 2.0: six 50 GB/s links, cache coherence with POWER9 (Summit, slide 26); NVSwitch (2018) connects 16 GPUs (slide 25). CUDA 9 adds cooperative groups and WMMA.

Tensor cores: a 4×4×4 matrix multiply-add per clock

$D = AB + C$ on 4×4 tiles, FP16 in and FP32 out: 64 fused multiply-adds per clock per tensor core, 512 per SM.

one tensor-core operation per clock



each of the 16 outputs is a 4-term dot product, multiplied exactly and summed in FP32: 64 FMAs per clock, against 16 for the quarter-SM's FP32 lanes

Why it is eight times faster

One instruction, 64 multiply-adds, and the operands move once: the tiles are read from registers, the products summed in a fixed adder tree, the sums written back; no per-element fetch, decode or register traffic (slide 13). Eight tensor cores per SM: $8 \times 64 \times 2$ FLOP $\times$ 80 SMs $\times$ 1.53 GHz $=$ 125 TFLOP/s (derived); the FP32 lanes manage 15.7 on the same clock.

How it is used, and what it costs

Programming: a warp cooperates on a 16×16×16 tile through the `wmma` API (CUDA 9): load fragments, `mma_sync`, store; or let cuBLAS and cuDNN pick tensor-core kernels (Markidis et al. 2018 measure the API).

Precision: FP16 inputs (10-bit mantissa), exact products, FP32 accumulation. Mixed-precision training (Micikevicius et al. 2017) keeps FP32 master weights, runs forward and backward in FP16 on tensor cores, and scales the loss so small gradients survive FP16's range: no loss of accuracy on image, speech and translation models†.

Feeding it: 125 TFLOP/s against 900 GB/s is a ridge of 139 FLOP/B (slide 15); only large-tile matrix multiply gets there, so the rest of a network (activations, normalization, softmax) is memory-bound and the tensor cores idle.

Independent threads, and what microbenchmarks measure

Volta gives each thread a program counter; measured latencies show where the cycles go in a modern SM.

divergent lanes that must synchronize

Pascal: one PC per warp; path A runs to the end, then path B



if A waits for a flag that B sets, the warp never reaches B: deadlock

Volta: a PC and stack per thread; the scheduler interleaves A and B



starvation-free; `__syncwarp()` reconverges when the programmer asks

Why it mattered

Pre-Volta code that assumed lock-step execution within a warp could break; code that needed a lock between threads of one warp could finally be written. Cooperative groups (CUDA 9) name the sets of threads that synchronize, from a few lanes to the whole grid.

Measured on a V100 (Jia et al. 2018)

Latencies in clock cycles: shared memory 19; L1 data hit 28; L2 hit about 193; global memory about 375 with a TLB hit and 1,029 with a TLB miss; an FP32 FMA 4.

What the numbers teach: a warp that misses to DRAM is away for 400 to 1,000 cycles, during which its SM issues from other warps (slide 12); an L2 hit costs 50 FMAs' worth of latency, so "cached" is not "free"; the L1 at 28 cycles is slower than a CPU's (4–5) because it serves 2,048 threads through a crossbar.

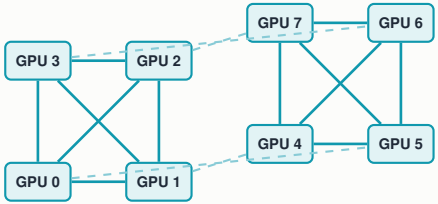
Register file: 256 KB per SM, banked, read in one cycle; register allocation is the first performance tool.

Method: pointer chasing and dependent-instruction chains timed with the clock register, as Wong et al. (2010) did on GT200 (global memory then: about 440 cycles; shared 38).

Beyond PCIe: NVLink, the DGX-1 cube, NVSwitch

Data-parallel training and multi-GPU HPC made the link between GPUs a first-class part of the architecture.

DGX-1 (2016): eight P100s in a hybrid cube mesh of NVLink; PCIe to the CPUs



solid: three of a P100's four links, within its group of four; dashed: the fourth, to the other group; 160 GB/s per GPU, two hops at most

Why not PCIe

PCIe 3.0 x16 moves 16 GB/s each way and shares a root complex with everything else; an all-reduce of a 100 MB gradient across eight GPUs costs tens of milliseconds per step on it.

The numbers†

PCIe 3.0 x16 (2012): 15.75 GB/s per direction.

NVLink 1.0 (P100, 2016): 20 GB/s per direction per link, four links: 160 GB/s per GPU. POWER8 CPUs had NVLink to the GPUs; x86 did not.

NVLink 2.0 (V100, 2017): 25 GB/s per direction, six links, 300 GB/s per GPU; cache coherence and address translation with POWER9, so CPU and GPU share memory (Summit, slide 26).

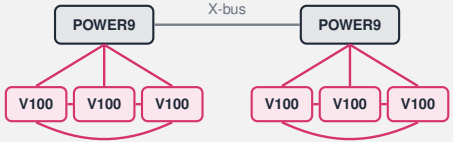
NVSwitch (DGX-2, March 2018): an 18-port crossbar, 50 GB/s per port; twelve of them give 16 V100s full NVLink bandwidth to any other, 2.4 TB/s of bisection bandwidth, and 512 GB of HBM2 as one address space.

For training: with the links, eight GPUs behave like one large one for data-parallel SGD (NCCL ring all-reduce); without them communication sets the step time.

GPUs in supercomputers: Tianhe-1A to Summit

Eight years from the first GPU-accelerated #1 to a machine where 97% of the FLOP/s sit on 27,648 V100s.

a Summit node (2018): two POWER9 CPUs, six V100s, NVLink 2.0 throughout



per node: about 42 TFLOP/s FP64 on the GPUs (ORNL; 47 at boost clock) against about 1 on the CPUs (derived); 512 GB DDR4, 96 GB HBM2, 1.6 TB NVMe, dual-rail EDR

Summit (ORNL, 2018)†

4,608 nodes, 27,648 V100s, 200 PFLOP/s peak; 122.3 PFLOP/s on HPL in June 2018 (TOP500 #1), 148.6 by June 2019; 13 MW design power. Sierra (LLNL), the same design with four GPUs per node, was #2 and #3. Both came from the 2014 CORAL procurement, the first time the U.S. labs bet their largest systems on GPUs.

The path there†

Tianhe-1A (NUDT, Nov 2010): 7,168 Tesla M2050s, 2.57 PFLOP/s HPL, #1; TSUBAME 2.0 and Nebulae on the same Fermi die.

Titan (ORNL, Nov 2012): Cray XK7, 18,688 K20Xs beside 16-core Opterons, 17.6 PFLOP/s HPL, #1; the first large machine whose applications had to be ported to CUDA or OpenACC to reach most of the peak.

Piz Daint (CSCS, 2016–17): Cray XC50, 5,320 P100s, 19.6 PFLOP/s HPL, #3 in November 2017.

What changed in the applications: by 2018 the codes on these machines (molecular dynamics, climate, QCD, cosmology, materials) had GPU versions; the port took most of a decade, and the memory system of slide 14 is where the work went. The U.S. exascale systems that followed are GPU machines.

The alternatives, and why they lost the decade

AMD's GCN and Vega, Intel's Larrabee and Xeon Phi, and the "100×" argument: software and perf/W decided it.

AMD

GCN (2012): a SIMT design close to Fermi's (64-wide wavefronts, a scalar unit, caches); the FirePro S9150 (2014; 2.5 TFLOP/s FP64, more than the K40) had the hardware; Vega's Radeon Instinct MI25 (2017) had 12.3 TFLOP/s FP32 and HBM2. **Software:** Close To Metal (2006), Stream, OpenCL, then ROCm and HIP (2016); by then CUDA had a decade of libraries and users.

Intel

Larrabee (2008): many small x86 cores with 16-wide vector units and coherent caches, meant to run graphics in software; cancelled as a GPU in December 2009. **Xeon Phi:** the same idea as a coprocessor (Knights Corner, 2012; about 1 TFLOP/s FP64) and a self-booting CPU (Knights Landing, 2016; up to 72 cores, 3.5 TFLOP/s FP64, 16 GB of MCDRAM at 490 GB/s); ended 2018.

The arguments of the decade

"100×": early GPGPU papers compared tuned GPU code with untuned CPU code. Lee et al. (ISCA 2010, Intel) tuned both: GTX 280 against Core i7-960 came out 2.5× on average over 14 kernels. The honest number is the ratio of peak bandwidths and FLOP/s, 3–5×, when both sides are done well.

Why GPUs still won: the 3–5× was real and compounded every generation; perf/W favored wide, slow lanes (slide 13); the memory was built for bandwidth; gamers paid for the dies.

Why it was NVIDIA's decade: CUDA from 2007 and its libraries; a Tesla line with ECC and FP64 before any competitor had both; a compute die every two years; every deep-learning framework on cuDNN by 2015.

What stayed constant for a decade, and what changed

From G80 to Volta the programming model and the SM template held; the counts, memory and datapaths moved.

Constant

1. The model: grids of blocks of threads, blocks on SMs, 32-thread warps, shared memory and barriers inside a block (slides 9–10). CUDA code from 2007 runs on a V100.
2. SIMT execution with masks and a warp scheduler (slide 11).
3. Latency hidden by resident warps and registers, not by large caches or out-of-order issue (slide 12).
4. The SM template: register file, schedulers, FP32 lanes, load/store units, SFUs, on-chip SRAM (slides 10, 21).
5. A device with its own memory, driven from a host (slides 9, 20).
6. The constraint: energy per operation, so data movement and instruction overhead are what every change attacks (slide 13).

Changed

Scale: 128 to 5,120 lanes, 0.68 to 21.1 billion transistors, 86 to 900 GB/s, at clocks between 0.75 and 1.53 GHz (slide 18).

Generality, then specialization: FP64, caches, ECC and pointers (2008–10) made it a processor; FP16 and tensor cores (2016–17) made part of it a matrix engine again.

Memory technology: GDDR3 to GDDR5 to HBM2 on an interposer; a unified virtual address space with page migration.

The interconnect: PCIe to NVLink to NVSwitch; the GPU stopped being a peripheral and became the node.

Thread semantics: one program counter per warp to one per thread (Volta).

The customer: graphics, then HPC, then deep learning; by 2017 the compute die was designed around the third.

In practice: reading a spec sheet, estimating, measuring

First-order performance of a GPU kernel follows from four numbers on the datasheet and one on the kernel.

Reading a spec sheet
Find five numbers: SM count and lanes per SM (peak FLOP/s $=$ SMs $\times$ lanes $\times$ 2 $\times$ clock), memory bandwidth, memory capacity, registers and shared memory per SM, and the host link. Check which precision the peak refers to and whether it is a boost clock. Divide peak FLOP/s by bandwidth: the ridge point tells you which of your kernels can use the arithmetic at all (slide 15). The rest of the sheet (texture rate, ROPs, pixel fill) is graphics.

Estimating a kernel
Count the bytes a kernel must move from DRAM once and the FLOPs it does; the ratio is its intensity. Left of the ridge, time $\approx$ bytes / bandwidth; right of it, FLOPs / peak. Then ask whether it can reach the roof: enough blocks to fill every SM several times over, coalesced loads (slide 14), no divergence (slide 11), enough warps or ILP to cover hundreds of cycles of latency (slide 12), and few enough registers per thread for that occupancy. A kernel at 60–80% of its roofline is done.

Measuring
Use the profiler (nvprof, then Nsight Compute): achieved occupancy, DRAM throughput against peak, L2 hit rate, warp stall reasons, divergence. Compare measured bytes to compulsory bytes; a ratio above 1 means uncoalesced or repeated loads. Compare achieved FLOP/s to the roofline at the measured intensity, not to peak. Microbenchmark what the datasheet omits (latencies, bank conflicts: Wong et al. 2010, Jia et al. 2018). Report the GPU, clocks, precision and problem size with every number.

Summary: what to take from this lecture

A GPU is a throughput machine: many simple lanes, many threads in flight, a wide memory, and a model that matches.

Eight things to remember

1. The work was always regular and independent.
2. Fixed function became programmable, unified, then general (G80, 2006); general grew a matrix unit (Volta, 2017).
3. SIMT: one instruction per 32 lanes, each lane a thread; divergence costs time, coalescing saves bytes.
4. Latency is hidden by threads and registers, not caches.
5. Energy set the design: amortize instructions, keep data close.
6. The roofline says what a kernel can get; the ridge rose from 4 to 139 FLOP/B.
7. Fermi made it a processor; Pascal and Volta a learning engine.
8. Software decided the decade: CUDA, libraries, frameworks.

Questions the next lecture inherits

How far can the tensor core go? Volta's $4\times4\times4$ tile became larger tiles, more formats and sparsity; the ridge keeps rising and the rest of the program is memory-bound.

Where does the memory go? 900 GB/s against 125 TFLOP/s is 0.007 bytes per FLOP; the stacks got taller and faster, and the problem did not go away.

Is the GPU still a device? NVLink coherence with POWER9 and NVSwitch made the node the unit; the host-device model of slide 9 is the part of the picture under most pressure.

Is 32 the right warp? AMD chose 64 and later offered 32; the choice trades divergence cost against instruction overhead.

The lecture notes expand every slide, with exercises and a reading list of the primary papers and whitepapers.

Sources

Architecture papers and whitepapers first, then history, then the systems and the debates; † marks vendor material.

Architecture papers

Lindholm, Nickolls, Oberman, Montrym, “NVIDIA Tesla: a unified graphics and computing architecture,” *IEEE Micro* 2008. Nickolls, Buck, Garland, Skadron, “Scalable parallel programming with CUDA,” *ACM Queue* 2008. Nickolls and Dally, “The GPU computing era,” *IEEE Micro* 2010. Wittenbrink, Kilgariff, Prabhu, “Fermi GF100 GPU architecture,” *IEEE Micro* 2011. Foley and Danskin, “Ultra-performance Pascal GPU and NVLink interconnect,” *IEEE Micro* 2017. Choquette, Giroux, Foley, “Volta: performance and programmability,” *IEEE Micro* 2018. Dally, Keckler, Kirk, “Evolution of the graphics processing unit,” *IEEE Micro* 2021. Keckler et al., “GPUs and the future of parallel computing,” *IEEE Micro* 2011. Garland and Kirk, “Understanding throughput-oriented architectures,” *CACM* 2010. Fatahalian and Houston, “A closer look at GPUs,” *CACM* 2008. Owens et al., “GPU computing,” *Proc. IEEE* 2008; Owens et al., survey, *Computer Graphics Forum* 2007. NVIDIA whitepapers†: Fermi (2009), Kepler GK110 (2012), GeForce GTX 980 (2014), Tesla P100 (2016), Tesla V100 (2017); Tesla datasheets; Hot Chips 28 and 30 talks.

Graphics history

Clark, “The Geometry Engine,” SIGGRAPH 1982. Akeley, “RealityEngine graphics,” SIGGRAPH 1993. Lindholm, Kilgard, Moreton, “A user-programmable vertex engine,” SIGGRAPH 2001. Mark et al., “Cg,” SIGGRAPH 2003. Montrym and Moreton, “The GeForce 6800,” *IEEE Micro* 2005. Andrews and Baker, “Xbox 360 system architecture,” *IEEE Micro* 2006. Blythe, “The Direct3D 10 system,” SIGGRAPH 2006. Buck et al., “Brook for GPUs,” SIGGRAPH 2004. Peddie, *The History of the GPU* (3 vols., 2022–23).

Microarchitecture, performance, energy

Wong et al., “Demystifying GPU microarchitecture through microbenchmarking,” ISPASS 2010. Mei and Chu, *IEEE TPDS* 2017. Jia, Maggioni, Staiger, Scarpazza, “Dissecting the NVIDIA Volta GPU architecture via microbenchmarking,” arXiv 2018. Volkov and Demmel, SC 2008; Volkov, GTC 2010† and PhD thesis 2016. Williams, Waterman, Patterson, “Roofline,” *CACM* 2009. Hong and Kim, ISCA 2009. Bakhoda et al., GPGPU-Sim, ISPASS 2009. Fung et al., MICRO 2007. Rogers et al., MICRO 2012. Lee et al., “Debunking the 100X GPU vs. CPU myth,” ISCA 2010. Esmaeilzadeh et al., “Dark silicon,” ISCA 2011. Dennard et al., *JSSC* 1974. Markidis et al., “NVIDIA tensor core programmability, performance and precision,” 2018. Micikevicius et al., “Mixed precision training,” 2017. Harris, “Optimizing parallel reduction in CUDA,” 2007†. Hennessy and Patterson, *Computer Architecture*, ch. 4. Kirk and Hwu, *Programming Massively Parallel Processors*.

Deep learning, systems, alternatives

Krizhevsky, Sutskever, Hinton, NIPS 2012. Chetlur et al., “cuDNN,” 2014. Raina, Madhavan, Ng, ICML 2009. Steinkraus, Simard, Buck, ICDAR 2005. Seiler et al., “Larrabee,” SIGGRAPH 2008. TOP500 lists 2010–2019†; ORNL Summit and Titan pages†; LLNL Sierra†; CSCS Piz Daint†; NVIDIA, AMD, Intel and Khronos press releases†; JEDEC HBM2 (JESD235A) and GDDR5X†. Full list with links in the paper kit.